

OSS-Next — Blockers Report

2026-07-24 · Branch `integrate-wp-render`

What is blocked, who can unblock it, and exactly what is needed.

One backend fix now unblocks a working purchase: the Django checkout crash (B1). As of this report, both front-end payment defects are closed and verified against the real Braintree sandbox, and credentials are configured. B1 is the last thing standing between the current build and an end-to-end order — and three further items sit behind it.

Closed since the 2026-07-23 report — no longer blockers:

- **Client-controlled charge amount** — the payment endpoint now recomputes the total server-side and never charges the browser's figure. The obvious follow-up (tampering item prices) was verified dead too: the backend prices from `product_id` . Confirmed end-to-end in the sandbox — a \$0.01 attack is refused (409), an honest purchase charges the correct \$3,425.50, AVS returns M/M.
- **Charged-but-no-order** — a charge that fails to record now shows a dedicated "payment received, order needs attention" screen with the transaction reference, instead of a raw error next to a re-chargeable button.

These were *frontend* fixes. The remaining checkout blocker (B1) is entirely backend.

Owner	Open items	Nature
Backend team	4	Code fixes and one ownership question
Client	5	Business/product decisions; no engineering blocked behind them
Frontend	0	Nothing actionable outstanding

Backend Blockers

B1 · CRITICAL Checkout endpoint crashes on every call

POST `/api/orders/checkout` returns 500 on every single request:

```
ValueError: Currency formatting is not possible using the 'C' locale.  
app/orders/views.py, line 89  
    locale.currency(float(item.total), grouping=True)
```

The server process has no locale configured, so Python's `locale.currency()` always raises. **The order record is saved before the crash** — real orders were confirmed created despite every attempt returning 500. With payment now working, a customer would have their card charged, have an order created, and still be shown a failure.

What we need: call `locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')` at startup, or drop `locale.currency()` and format the amount directly (e.g. `f"${amount:,.2f}"`). The second is more robust — it doesn't depend on locales being installed in the container image.

B2 Order status never becomes `paid`

We send `payment_method: "braintree"` and the Braintree `transaction_id` with every checkout, but the order status doesn't move to `paid`. Something else server-side governs it, and we can't see what.

What we need: confirm what sets an order to `paid`, and whether `transaction_id` is actually persisted. If it's discarded, tracing a payment from an order back to Braintree won't be possible.

B3 The pages API serves malformed CSS

The CSS minifier strips `/* */` delimiters but leaves the comment text in place, and emits unbalanced braces:

```
once slick adds its class,show it smoothly .shipping-container-slider...{...}  
}.slick-slide:not(.slick-active){opacity:0!important}
```

Strict parsers reject this outright. **Browsers error-recover by discarding every rule after the fault** — so the live WordPress site is silently losing styling today, not only our app. We work around it with a tolerant parser, but the source is wrong.

What we need: fix the minifier in the Django CSS pipeline so comments are removed whole and braces stay balanced.

B4 Who sends order confirmation emails?

There's no email-sending code anywhere in the Next.js application — no SendGrid, nodemailer, or SMTP. If confirmation emails are being sent today, it's entirely a Django responsibility and invisible from our side.

What we need: a yes/no on whether Django sends order confirmation emails, and on which events.

Client Decisions

None of these block engineering — they block *starting* it. Two carry active customer-facing risk (C1, C2), and both have a cheap interim mitigation that needs no product decision.

C1 · CUSTOMER-FACING RISK The homepage quote form silently discards submissions

The form shows "✓ **Quote Request Sent — We'll be in touch!**" while the customer's name, email, phone, ZIP and container details go **nowhere**. Every lead submitted through it is lost, and the visitor believes they've contacted the business.

What we need: the Zoho Forms account and form details, to wire it to the real destination. **Until then, consider disabling the form or removing the success message** — the current behaviour is worse than no form at all.

C2 · CUSTOMER-FACING RISK The site advertises live chat that doesn't exist

The homepage trust strip promises "Phone, Email & Chat" support. There's no chat feature anywhere, and none planned yet.

What we need: either a decision to build chat, or approval to change the copy. **The copy fix takes minutes and needs no engineering decision** — we recommend it now regardless of the longer-term answer.

C3 Newsletter signup UI

The largest genuinely unbuilt feature. The backend calls already exist on our side, so each piece is "build the UI and call this": a footer widget, an account-dashboard toggle, and an unsubscribe page.

What we need: confirmation to build it, and where the signup should appear.

C4 Address book — one address, or many?

Today a customer has exactly one billing and one shipping address on their profile. No add/remove, no multiple saved addresses, no default selection. Fine as "edit your address"; not an address book.

What we need: a decision on whether multiple saved addresses are wanted. This also needs backend support — the profile endpoint has no concept of more than one.

C5 Shipment tracking in order history

Order history shows real status (pending / paid / shipped / delivered), but there's no carrier or tracking number anywhere in the data, so customers can't track a delivery.

What we need: confirmation that tracking numbers are wanted, and whether the backend can begin returning a carrier + tracking number on the order record.

Waiting Behind B1

Not separate work — these unblock themselves once the checkout crash is fixed:

Item	Why it's stuck
End-to-end purchase verification	Front end is built and verified; the endpoint 500s.

Populated order-history rendering	Built, but no order can be created to test it against.
Cart-close on successful checkout	Wired, but never reached, because checkout never completes.

Known and Deliberate — No Action Requested

Product-listing images don't set a priority hint. The cards render through the search library's own list component, which doesn't expose an item's position, so marking the first card would mean replacing that component with a hand-rolled version and risking pagination and empty-state regressions — for a minor performance hint on content rendered after page load anyway. Recorded so it isn't re-reported as an oversight.